

Ecosystem Survival: First Design Proposal for a Grid-Based Game Driven by Greedy and Backtracking Algorithms

Cristian Esteban Castañeda Vargas, Julian Carvajal Garnica, Juan Pablo Angulo Guerrero
Computer Science I
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Abstract—Grid-based educational videogames provide an effective way to integrate algorithmic thinking and data structure concepts into practical learning experiences. This paper proposes *Ecosystem Survival*, a turn-based game in which players protect endangered species using a greedy strategy to prioritize feeding decisions and a backtracking strategy to compute safe migration routes, supported by C++ data structures and a JSON-based communication bridge. The resulting design defines clear game rules, a modular system architecture, and preliminary algorithmic analyses that establish a solid foundation for later implementation and evaluation.

Index Terms—greedy algorithms, backtracking, AVL tree, linked list, grid-based game, Pygame, JSON bridge, algorithm design

I. INTRODUCTION

Algorithm design and data structure analysis are core components of introductory computer science education. However, these topics are often perceived by students as abstract and disconnected from real-world applications. Educational videogames offer an opportunity to bridge this gap by embedding classical algorithms directly into interactive decision-making processes, allowing learners to observe how algorithmic strategies influence system behavior over time.

Within this context, grid-based games provide a particularly suitable environment for modeling constrained problems involving movement, resource management, and priority-based decisions. Such environments naturally lend themselves to greedy heuristics, which make locally optimal choices under limited information, as well as backtracking techniques, which are useful for exploring multiple alternatives when constraints must be satisfied simultaneously. When combined with explicit data structures for state management, these techniques form a coherent framework for demonstrating fundamental computer science concepts in practice.

The project presented in this paper, *Ecosystem Survival*, adopts this approach by modeling wildlife preservation as a turn-based strategy game. Players must manage limited resources, evaluate risk levels among species, and plan safe migration routes within a constrained grid. The design explicitly aligns each gameplay mechanic with a specific algorithmic or data-structural concept studied in the Computer Science I course, ensuring that the game mechanics are not merely decorative but represent the underlying computational logic.

This document introduces the initial design decisions of the project. Rather than focusing on implementation details, it establishes the problem domain, the proposed solution strategy, and the architectural foundations that will guide subsequent development and evaluation phases.

II. PROJECT CONTEXT AND VARIANT DEFINITION

The *Ecosystem Survival* project is developed within the framework of the Computer Science I course, which emphasizes the integration of algorithm design and data structures through a unified software artifact. All teams participating in the course share a common technical architecture: a core engine implemented in C++ is responsible for managing the required data structures and game state, while a Python layer handles algorithmic decision-making and user interaction. Communication between these layers is strictly defined through JSON file exchange to ensure loose coupling and clear responsibility boundaries.

Each team is assigned a specific game variant that determines the thematic context as well as the required combination of algorithms and data structures. The variant addressed in this project models an ecosystem preservation scenario, where the player must manage multiple species under environmental constraints. This setting directly maps to the required computational components: a greedy strategy is used to prioritize feeding decisions for endangered species, while a backtracking algorithm is employed to compute valid migration routes toward safe zones under spatial restrictions.

In addition, the variant mandates the explicit use of a linked list and a self-balancing tree implemented from scratch in C++. The linked list naturally represents migration paths as ordered sequences of positions, while the AVL tree is well suited for managing species population data that must be searched and updated efficiently over time. These requirements guide the overall system design and ensure that the project remains aligned with the learning objectives of the course.

The design choices presented in this paper were made with the goal of satisfying all variant constraints while maintaining conceptual clarity and implementation feasibility. By tightly coupling gameplay mechanics with their underlying algorithmic and data-structural foundations, the project ensures that

every user action has a clear computational interpretation, reinforcing the educational purpose of the game.

III. GAME OVERVIEW

A. Objective

The objective of *Ecosystem Survival* is to model decision-making under constrained resources in a dynamic environment. The player is responsible for preventing the extinction of multiple species by managing hunger levels and population sizes while guiding species toward designated safe zones. The game is won if all species successfully reach a safe zone before any population reaches zero. Conversely, the game ends in defeat as soon as at least one species becomes extinct.

This objective reflects a prioritization problem under limited actions, where not all species can be attended simultaneously. As a result, the player must continuously evaluate risk and make informed decisions based on algorithmic feedback.

B. Board Configuration

The game environment is represented as a two-dimensional 8×8 grid. At the start of each game, the board is initialized under spatial constraints that ensure no two entities occupy the same cell. The grid contains multiple interacting elements, including species, food resources, predator zones, and safe areas, all of which influence valid actions and algorithmic outcomes during gameplay.

The grid-based representation enables a clear mapping between spatial constraints and algorithmic exploration. Each cell represents a discrete state that may be evaluated during movement planning and validation, making the board a suitable abstraction for pathfinding via backtracking.

C. Turn Structure

Gameplay progresses in discrete turns. During each turn, the player is allowed to perform exactly two actions, which may consist of feeding a species, migrating a species, or a combination of both. This strict action limit introduces a trade-off between short-term survival, managed through feeding, and long-term safety, achieved through migration.

By limiting the number of actions per turn, the game enforces the need for prioritization and reinforces the role of the greedy recommendation algorithm as a decision-support mechanism rather than an automatic controller.

D. Hunger and Population Dynamics

Each species is characterized by a hunger level and a population count. Hunger increases incrementally at the end of every turn, representing ongoing resource consumption. If the hunger level of a species reaches a critical threshold, the species begins to lose population over subsequent turns until hunger is reduced.

Food resources placed on the grid allow hunger to be decreased when a species is fed directly or when it migrates across a food cell. Once consumed, food resources are removed from the board. Population loss is irreversible; once a

population reaches zero, the species is marked as extinct, and the game terminates immediately.

These dynamics ensure that feeding and movement decisions are tightly coupled and that delayed action can have irreversible consequences, reinforcing the need for algorithmic guidance.

IV. DESIGN DECISIONS

This section documents the main technical decisions adopted during the initial design phase of the project. These decisions were guided by clarity, ease of debugging, alignment with course requirements, and suitability for algorithmic explanation during evaluation.

A. Board Representation and Symbols

The game board uses explicit character symbols to represent entities rather than numeric encodings. This choice simplifies visualization during debugging, improves readability of the game state when inspecting JSON files, and facilitates explanation during demonstrations and evaluations.

Each cell in the grid stores a single symbol representing its current content, such as an empty space, a species, a predator cell, a food resource, or a safe zone. By using human-readable symbols, the board can be directly interpreted without additional translation layers.

TABLE I: Board symbols

Symbol	Meaning
[]	Empty cell
R	Rabbit
F	Fox
T	Turtle
D	Deer
X	Danger / predator cell
C	Food cell
Z	Safe zone

B. Movement Model

Species migration is allowed in eight directions: horizontal, vertical, and diagonal. Allowing diagonal movement increases the branching factor of the search space and makes the backtracking process non-trivial, which is desirable for demonstrating recursive exploration and constraint validation.

Movement rules were designed to enforce meaningful constraints. A species may not move outside the board, enter predator cells, or pass through cells occupied by other species. Empty cells, food cells, and safe zones are traversable. Safe zones act as terminal states and may be shared by multiple species, while non-safe cells remain exclusive to preserve spatial constraints during search.

C. Species State Representation

Each species is modeled as a structured entity with fields for identification, dynamic state, and spatial position. The selected attributes are the species symbol and name for readability, population and hunger for survival dynamics, position for spatial reasoning, and status for global outcome evaluation.

The `status` field allows the engine and the UI to quickly determine whether a species is active, safe, or extinct. This avoids repeated conditional checks on population or position during gameplay and simplifies end-condition validation.

Overall, these design decisions aim to produce a clean and explainable system where game mechanics map directly to computational concepts, supporting both implementation correctness and pedagogical clarity..

V. SYSTEM ARCHITECTURE

The architecture of *Ecosystem Survival* is organized into three well-defined logical layers, each with clearly separated responsibilities. This layered approach ensures modularity, simplifies debugging, and facilitates independent development of the system components while complying with the architectural constraints established by the course.

The core of the system is the C++ engine, which acts as the authoritative source of the game state. This layer is responsible for implementing and maintaining the required data structures, namely a linked list for migration paths and an AVL tree for managing species population data. The engine processes player actions, enforces all game rules, updates species attributes, and guarantees consistency of the global state across turns. Player actions are read from `input.json`, and after processing, the updated state is written to `state.json`.

Algorithmic reasoning is implemented in the Python layer. This layer includes two independent modules: a greedy algorithm that computes which species should be prioritized for feeding based on risk evaluation, and a backtracking algorithm that searches for valid migration routes toward safe zones while respecting spatial constraints. These algorithms operate exclusively as decision-support mechanisms. They analyze the current game state but do not directly modify it, preserving the separation between logic evaluation and state mutation.

The third layer is the Python-based user interface, implemented using Pygame. The interface renders the grid-based board, displays relevant information such as species population, hunger levels, and algorithmic recommendations, and collects player input. To avoid tight coupling with the C++ engine, the interface communicates indirectly through a dedicated bridge module. This bridge is responsible for writing player actions to `input.json` and reading the updated state from `state.json` for rendering and further algorithmic analysis.

Figure 1 presents an overview of the system architecture and illustrates the flow of information between the C++ engine, the Python algorithms, the user interface, and the JSON-based communication bridge. By explicitly separating responsibilities and relying on file-based interaction, the architecture promotes clarity, reproducibility, and robustness, making it well suited for both educational purposes and incremental development.

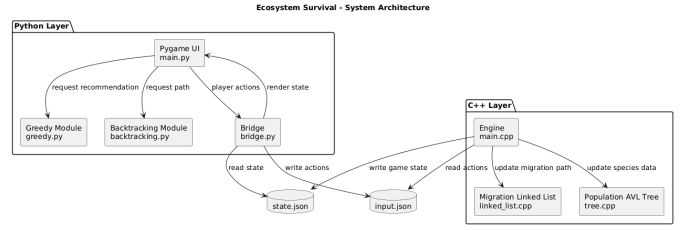


Fig. 1: Proposed system architecture of the project and data flow between components.

VI. GREEDY STRATEGY

A. Motivation

One of the central challenges in *Ecosystem Survival* is deciding which species should be prioritized for feeding when actions are limited. Since not all species can be attended simultaneously, the game requires a systematic method to identify the most critical cases at each turn. A greedy approach is well suited for this purpose, as it allows decisions to be made based on local information while maintaining low computational overhead.

The greedy strategy does not aim to optimize long-term global outcomes. Instead, it provides an immediate and interpretable recommendation that reflects the current risk faced by each species. This design aligns with the educational goal of the project, as it allows players to understand and reason about the heuristic being applied rather than treating algorithmic decisions as opaque outcomes.

B. Risk Evaluation Function

To determine which species should be fed first, the game defines a numerical risk value that captures both urgency and vulnerability. Urgency is represented by the hunger level of a species, while vulnerability is associated with its population size. The proposed risk function is defined as

$$\text{risk} = \text{hunger} + (10 - \text{population}) \quad (1)$$

This formulation ensures that species with high hunger levels or low population sizes receive higher risk scores. By combining both factors into a single metric, the algorithm can compare species directly and produce a clear ranking at each turn.

Only species that are currently active are considered during evaluation. Species already located in safe zones or marked as extinct are excluded from the recommendation process, as they no longer require feeding actions.

C. Greedy Recommendation Process

The greedy algorithm iterates once over the list of species stored in the current game state. For each valid species, the risk value is computed using the defined function and compared against the highest risk observed so far. The species with the maximum risk score is selected as the recommended candidate for feeding.

This process produces exactly one recommendation per turn and does not alter the game state. The result is exposed to the user interface and displayed as advisory information. The final decision of whether to follow the recommendation remains under player control, preserving interactivity and allowing exploration of alternative strategies.

D. Greedy Pseudocode

Listing 1: Greedy recommendation pseudocode

```
function GreedyRecommend(species_list):
    best_species <- null
    highest_risk <- -1

    for each species in species_list:
        if species.status != "extinct" and species.
            status != "safe":
            risk <- species.hunger + (10 - species.
                population)

            if risk > highest_risk:
                highest_risk <- risk
                best_species <- species

    return best_species
```

Figure 2 illustrates the logical flow of the greedy decision process, from risk evaluation to recommendation output.

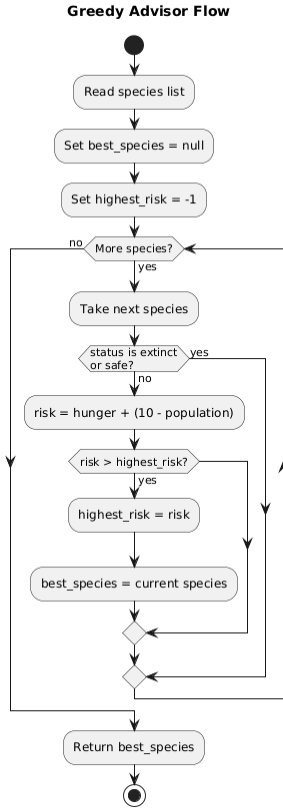


Fig. 2: Decision flow of the greedy recommendation algorithm.

E. Preliminary Complexity Analysis

Let s be the number of species in the game. The greedy recommendation algorithm performs a single pass over the species list and executes constant-time operations for each element. Therefore, the time complexity of the algorithm is

$$T(s) = O(s) \quad (2)$$

The algorithm uses a constant number of auxiliary variables and does not allocate additional memory proportional to the input size. As a result, its auxiliary space complexity is

$$S(s) = O(1) \quad (3)$$

This efficiency makes the greedy strategy suitable for execution at every turn without impacting responsiveness, even as species states change dynamically throughout the game.

VII. BACKTRACKING STRATEGY

A. Motivation

Migration is a fundamental mechanic in *Ecosystem Survival*, as species must reach safe zones to ensure long-term survival. Unlike feeding decisions, migration involves spatial constraints and multiple possible movement alternatives. A valid migration route may require exploring different paths, retreating from dead ends, and validating each step against several constraints. For this reason, a backtracking approach is well suited to model the search process.

Backtracking allows the algorithm to systematically explore the grid, testing candidate moves while maintaining the ability to undo decisions when constraints are violated. This makes it appropriate for constrained pathfinding problems in which feasibility is more important than optimality.

B. Search Model

The migration search is performed on the 8×8 grid that represents the game board. Starting from the current position of a selected species, the algorithm recursively explores neighboring cells in all eight possible directions: horizontal, vertical, and diagonal. Each recursive step represents a tentative move in the migration path.

A migration path is considered valid if it reaches any cell marked as a safe zone. If the algorithm reaches such a cell, the search terminates successfully and the constructed path is returned. If all possible moves from a given position lead to invalid states or dead ends, the algorithm backtracks by removing the last move from the path and exploring alternative directions.

C. Movement Constraints

To ensure correctness and prevent invalid routes, each candidate move must satisfy a set of constraints. A move is considered valid only if the target cell lies within board boundaries, has not been visited previously in the current path, and does not correspond to a predator cell. In addition, cells occupied by other species are treated as blocking entities and cannot be traversed.

These constraints are centralized in a validation function invoked before each recursive call. By separating validation logic from the search procedure, the algorithm remains clean and easier to reason about during analysis and debugging.

D. Backtracking Pseudocode

Listing 2: Backtracking migration pseudocode

```
function FindMigrationPath(board, row, col, visited,
    path):
    if board[row][col] == "Z":
        return true

    visited[row][col] <- true

    for each (dr, dc) in 8 directions:
        new_row <- row + dr
        new_col <- col + dc

        if IsValidCell(board, new_row, new_col,
            visited):
            append (new_row, new_col) to path

            if FindMigrationPath(board, new_row,
                new_col, visited, path):
                return true

            remove last element from path

    return false
```

E. Validation Function

The validation function ensures that all spatial and gameplay constraints are respected before a move is attempted.

Listing 3: Cell validation pseudocode

```
function IsValidCell(board, row, col, visited):
    if row < 0 or row >= 8:
        return false
    if col < 0 or col >= 8:
        return false
    if visited[row][col] == true:
        return false
    if board[row][col] == "X":
        return false
    if board[row][col] is another species symbol:
        return false

    return true
```

Figure 3 illustrates the logical flow of the backtracking search process, including recursive exploration and path roll-back.

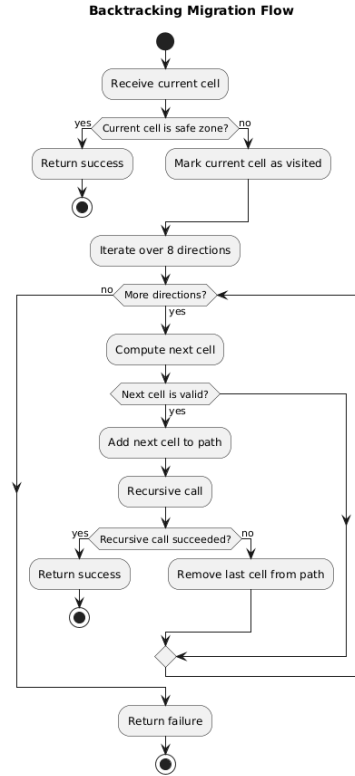


Fig. 3: Backtracking search flow for species migration toward a safe zone.

F. Preliminary Complexity Analysis

Let k denote the number of reachable cells explored during the migration search. In the worst case, the algorithm may attempt to explore multiple alternatives from each cell, with up to eight possible directions per step. As a result, the worst-case time complexity can be expressed as

$$T(k) = O(8^k) \quad (4)$$

This exponential behavior is inherent to exhaustive backtracking search. However, in practice, the effective search space is significantly reduced by board boundaries, obstacles, and visited-cell constraints.

The auxiliary space complexity is dominated by the recursion stack and the storage of the current migration path, both of which grow linearly with the depth of the search. Therefore, the auxiliary space complexity is

$$S(k) = O(k) \quad (5)$$

Given the fixed size of the board, these complexity bounds are acceptable for an academic prototype and allow the algorithm to run interactively without performance issues.

VIII. REQUIRED DATA STRUCTURES

The design of *Ecosystem Survival* requires the explicit implementation of two fundamental data structures in C++: a linked list and a self-balancing tree. These structures are

not auxiliary components, but integral elements of the game mechanics, as they directly support migration planning and population management. Their selection was guided by both functional suitability and course requirements.

A. Migration Path Linked List

Migration routes generated by the backtracking algorithm are represented in the C++ engine using a singly linked list implemented from scratch. Each node of the list stores a pair of coordinates corresponding to a grid position and a pointer to the next node in the path.

Listing 4: Conceptual linked list node for migration paths

```
struct PathNode {
    int row;
    int col;
    PathNode* next;
};
```

This structure is well suited to represent ordered sequences of positions, as migration paths are naturally linear and sequential. The linked list allows the engine to traverse the path efficiently and to update or discard the entire route when a migration action is completed. If a tail pointer is maintained, new nodes can be appended in constant time, which simplifies path construction after a valid route is found.

From a pedagogical perspective, using a linked list reinforces fundamental concepts such as dynamic memory management and pointer manipulation, which are core topics of the course.

Figure 4 illustrates the conceptual structure of the linked list used to store migration paths.

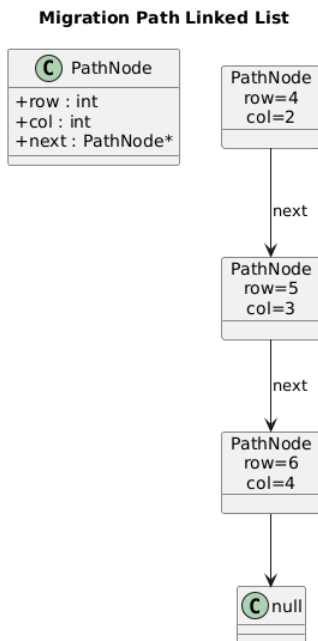


Fig. 4: Conceptual linked list structure representing a migration path.

B. Population AVL Tree

Species population data is stored and managed using an AVL tree implemented from scratch in C++. Each node of the tree represents a species and stores both identification and dynamic state attributes, including population and hunger values, along with the information required to maintain balance.

Listing 5: Conceptual AVL tree node for species data

```
struct AVLNode {
    string symbol;
    string name;
    int population;
    int hunger;
    int height;
    AVLNode* left;
    AVLNode* right;
};
```

An AVL tree was selected over a standard binary search tree in order to guarantee logarithmic performance for search, insertion, and update operations, regardless of insertion order. Since species population values may change frequently during gameplay, maintaining balance ensures predictable performance across turns. This is particularly relevant when updating species states or querying population data during game logic execution.

The AVL tree also serves an educational purpose by exposing students to self-balancing tree mechanics, including rotations and height maintenance, which are essential topics in advanced data structure design.

Figure 5 presents a conceptual representation of the AVL tree structure used in the project.

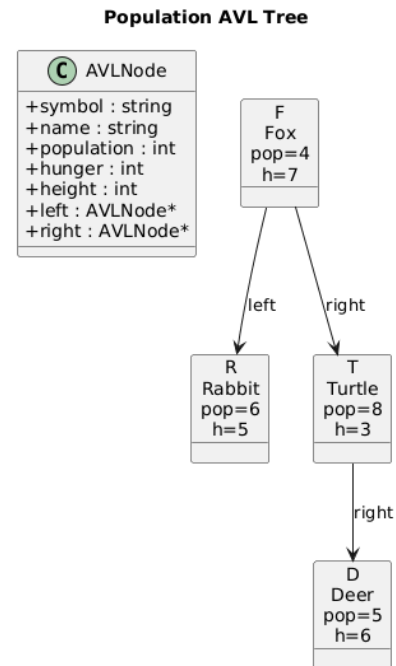


Fig. 5: Conceptual AVL tree used for species population management.

C. Data Structure Complexity Summary

The use of these data structures provides well-defined performance guarantees for the operations required by the game mechanics. Table II summarizes the preliminary time complexities of the main supported operations.

TABLE II: Preliminary complexity summary

Structure / Algorithm	Operation	Time Complexity
Linked list	Traversal	$O(n)$
Linked list	Append (with tail)	$O(1)$
AVL tree	Search	$O(\log n)$
AVL tree	Insert	$O(\log n)$
AVL tree	Update / Delete	$O(\log n)$

Together, these structures provide an efficient and conceptually clear foundation for the game engine, ensuring that migration paths and species population data are managed consistently and in accordance with the learning objectives of the course.

IX. JSON FILE I/O CONTRACT

The integration between the C++ engine and the Python layer is achieved exclusively through JSON file exchange. This design choice enforces loose coupling between components implemented in different programming languages and guarantees that each layer can be developed, tested, and executed independently.

Two JSON files define the communication contract: `input.json` and `state.json`. Their structure and semantics are fixed and shared by all components of the system.

A. Input File Schema

The `input.json` file is written by the Python bridge module and read by the C++ engine at the beginning of each action processing cycle. This file specifies the action selected by the player and the target species.

Listing 6: Example `input.json` for a feeding action

```
{
  "action": "feed",
  "species": "R"
}
```

For migration actions, the schema remains identical, with the action field indicating a different operation:

Listing 7: Example `input.json` for a migration action

```
{
  "action": "migrate",
  "species": "F"
}
```

The simplicity of this schema ensures that only one action is processed at a time and prevents ambiguity in engine execution.

B. Output File Schema

After processing a player action, the C++ engine writes the complete updated game state to `state.json`. This file is read by the Python layer for rendering, algorithmic evaluation, and user feedback.

Listing 8: Example `state.json`

```
{
  "grid_size": 8,
  "turn": 3,
  "actions_left": 1,
  "board": [...],
  "species": [...],
  "greedy_recommendation": "F",
  "migration_path":
    [[4,2],[5,3],[6,4],[7,5],[7,6],[7,7]],
  "game_status": "running"
}
```

This schema provides both a full grid snapshot for visualization and structured species data for algorithmic processing. By exposing the migration path explicitly, the interface can visually illustrate the result of the backtracking algorithm.

C. Design Rationale

Including the entire game state in `state.json` simplifies debugging and visualization, as the file can be inspected directly after each action. This explicit contract also ensures deterministic behavior and reproducibility, which are critical for evaluation and grading.

X. USER INTERFACE PROPOSAL

The user interface of *Ecosystem Survival* is implemented using the Pygame library and follows a grid-based visual layout. Its primary function is to provide a clear visual representation of the game state and to allow the player to interact with the system in a controlled and intuitive manner.

The interface is divided into three main regions. The central area displays the 8×8 grid, where each cell reflects its current content using graphical symbols. A side panel presents information about each species, including population, hunger level, and current status. An action panel allows the player to select available actions and displays advisory outputs generated by the greedy and backtracking algorithms.

Special emphasis is placed on visual feedback. Algorithmic recommendations and migration paths are highlighted on the grid to help the player understand how decisions are generated. This reinforces the educational objective of the project by making algorithmic reasoning visible rather than implicit.

Figure 6 presents an initial wireframe of the proposed interface layout.

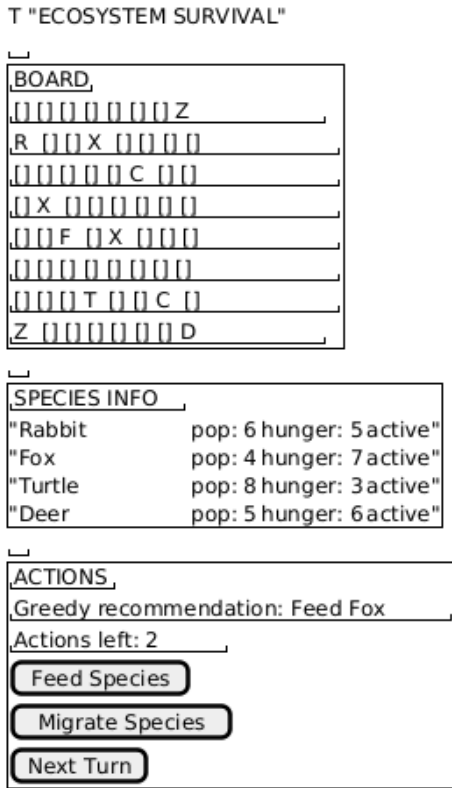


Fig. 6: Initial wireframe of the Pygame user interface.

XI. PLANNED TEAM WORK SPLIT

The project is developed by a three-member team, with responsibilities distributed according to the layered system architecture. While each member has a primary role, all team members are expected to understand the complete system for final evaluation and presentation.

TABLE III: Planned work distribution

Member	Role	Responsibilities
Cristian Esteban Castañeda Vargas	Engine Developer	Implementation of the C++ engine, linked list, AVL tree, and JSON state processing
Julian Carvajal Garnica	Algorithm Developer	Design and implementation of greedy and backtracking algorithms in Python
Juan Pablo Angulo Guerrero	UI Developer	Pygame interface development, bridge module, and visualization of algorithm outputs

XII. DISCUSSION

The proposed design aims to balance algorithmic clarity, system modularity, and gameplay feasibility. By constraining the board size and action count, the game creates meaningful decision points while keeping algorithmic complexity manageable for an academic prototype.

The explicit separation between state management, algorithmic reasoning, and user interaction simplifies both implementation and explanation. In particular, treating greedy and backtracking algorithms as advisory modules preserves player agency while making computational strategies observable and interpretable.

At the same time, the design remains flexible. The modular architecture allows additional rules, species, or heuristics to be incorporated incrementally without requiring structural changes to the engine or interface.

XIII. CONCLUSION

This paper presented the initial design of *Ecosystem Survival*, a grid-based educational videogame that integrates greedy algorithms, backtracking search, and fundamental data structures into its core mechanics. The project establishes a clear mapping between gameplay actions and computational concepts, ensuring that algorithms and data structures are not auxiliary components but central elements of the experience.

The defined architecture, game rules, algorithms, data structures, and JSON communication contract provide a solid foundation for subsequent implementation. Future work will focus on implementing the described components, integrating them into a playable system, and evaluating the effectiveness of the game as a learning tool during the final stages of the course.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Cham, Switzerland: Springer, 2020.
- [3] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014.
- [4] Pygame Community, "Pygame documentation," [Online]. Available: <https://www.pygame.org/docs/>
- [5] Python Software Foundation, "json — JSON encoder and decoder," [Online]. Available: <https://docs.python.org/3/library/json.html>
- [6] N. Lohmann, "JSON for Modern C++," [Online]. Available: <https://github.com/nlohmann/json>